## INF2004 Embedded Systems Programming

# Week 8 Tribunal 1 Template

*Architecture Defence & Integration Readiness | AI ideation, live update, AI-vetted gap check*

| Item | Details |
|---|---|
| Purpose | Defend Week 6 architecture, justify changes, and show readiness for partial integration. |
| Core workflow | Pre-submission AI ideation -> live Zoom-updated version -> final AI-vetted gap check |
| Assessment focus | Evidence, engineering judgement, negotiation, and ability to identify missing work |
| Submission format | Completed document plus Zoom recording and evidence links |

**Important: this submission has THREE versions inside one document**

- Version A: Pre-tribunal AI ideation. Each buddy uses AI to ideate their subsystem position before Zoom.
- Version B: Live Zoom-updated version. Design decisions are updated during the Zoom discussion through challenge, negotiation, and constraint response.
- Version C: Final AI-vetted gap check. After discussion, AI is used as a reviewer to identify missing assumptions, risks, tests, or unclear decisions.
- Marks are not awarded for blindly following AI. Marks are awarded for critical evaluation and human engineering judgement; meaning, I want to see you guys discuss during zoom.

## Team Information

| Item | Details |
|---|---|
| Group ID | LAB-P1 SE06 |
| Repository Link | https://github.com/adoreblvnk/whattheflip |

| Buddy | Student ID | Primary subsystem / persona |
|---|---|---|
| Tan Yan Qi | 2500030 | **Wired Protocol Auto-Discovery Engine:** 4-pin hardware edge timing via RMT RX, pulse histogram profiling, baud estimation (9600–115200 baud ±2%, I2C 100/400 kHz ±2%), and dynamic probe-to-protocol pin mapping (GP4–GP7). |
| Annie Goh | 2503469 | **Physical Chaos & Fault Injection Engine:** Frame-synchronized glitch injection (bit-flips, I2C ACK suppression, clock stretching), bus contention protection, and hardware E-stop driver release. |
| Zayne Lee | 2503844 | **Wireless Multi-Protocol Sniffer Engine:** ESP32-C6 2.4 GHz promiscuous radio capture for Wi-Fi 6 (802.11ax) management frames and IEEE 802.15.4 (Zigbee 3.0), extracting RSSI, channel, and timestamps. |
| Ryan Ke | 2501911 | **Telemetry Ingestion & PCAP Storage Engine:** Zero-copy fixed memory ring buffers, packet serialization into standard libpcap format (.pcap), and high-speed FAT32 MicroSD card logging. |
| Joseph Poon | 2501079 | **System Supervisor, Web UI & Victim Control:** uT-Kernel supervisor coordination, WebSocket live streaming dashboard over local Wi-Fi, hardware E-stop interrupt handling, NeoPixel/buzzer indicators, and victim stimulus control. |

## Version A: Pre-Tribunal AI Ideation

### Instructions

- Each buddy must use AI before Zoom session to prepare subsystem-specific arguments, risks, and negotiation points.
- The AI output must be critiqued. Identify what is useful, unrealistic, incomplete, or requires validation.
- For Tribunal 1, compare against the Week 6 architecture freeze. For Tribunal 2, compare against actual integration evidence.

| Buddy | AI prompt used | Useful AI ideas | Weak / unrealistic AI ideas | Position before Zoom |
|---|---|---|---|---|
| Tan Yan Qi | **Model:** Sonnet 5.5<br><br>**Prompt:** "I'm designing a Auto-Discovery Engine on ESP32-C6 which is using µT-Kernel 3.0 and 4 GPIO pins (GP04-GP07) are used as probes. We need to identify UART (9600-115200 baud) and I2C (100/400 kHz) within 500 ms without manually doing it. What RTOS architecture and hardware peripheral strategy should I use?" | Suggested using hardware edge timing (ESP32-C6 RMT RX peripheral) and load information directly to memory using DMA to reduce CPU usage.<br><br>Use pulse width histogram to cluster baud rates and determine whether it's UART or I2C.<br><br>Keep probe pins in High-Z floating mode during boot. | Suggested attaching a software GPIO interrupt ISR (`gpio_isr_handler_add`) to every pin edge and enqueueing timestamps into a message queue (`tk_snd_mbx`).<br><br>for 400 kHz I2C 800,000 edge interrupts/sec would overwhelm a 160 MHz CPU causing CPU lockup.<br><br>Hallucinated a non-existent ESP32-C6 hardware I2C auto-baud register. | Don't implement software GPIO interrupt ISR<br><br>Use hardware RMT RX peripheral offload instead because it captures pulse durations directly into RAM and thus using less CPU<br><br>`AutoDiscovery_Task` periodically polls and classifies pulse distributions using statistical bounds (e.g.<br><br>$8.68\mu s \pm 2\%$ for 115200 baud;<br><br>$2.5\mu s \pm 2\%$ for 400 kHz I2C). |
| Annie Goh | **Model:** GLM 5.3<br><br>**Prompt:**<br>Target microcontrollers are Raspberry Pi Picos.<br><br>An ESP32-C6 is running a micro t-kernel 3.0 with sub-500ns jitter.<br><br>How can deterministic physical bit-flip fault injection and I2C ACK suppression be implemented?<br><br>Implementation should ensure that driving the target line low does not | **Problem:**<br>Raised that Real Time Operation System has jitters that affect task scheduling. Jitters are unpredictable delays ranging from 1 to 5micros. A data bit width is 8.68ms at 115,200 baud. A jitter is almost as big as a bit. Software-timed GPIO toggles for precise injection unfeasible as the exact timing window gets missed by the jitter.<br><br>**Solution:** | **<u>Weak Solution 1</u>**<br>**Solution:**<br>Suggested using TXS0108E bidirectional level shifter for injection.<br><br>**Problem:**<br>Shifter slows down at the edge of a bit flipping to keep them clean. Adds 10 to 15ns of delay. Precise injection unfeasible as the exact timing window gets missed by 'cleaning'. Shifter also cannot handle fast open-drain signals properly. | **For physical bit-flip fault injection:**<br>Do not use TXS0108E.<br><br>Connect the ESP 3.3V to the victim TX pin with a 220ohm resistor for safety.<br><br>Use EMT via RMT to have less than 500ns jitter.<br><br>Keep injection brief and rare. Equal or less than 10micros and 5% duty cycle. Protects microchip from overheating and burn out. |

| | | | | |
|---|---|---|---|---|
| | burn the target microcontrollers. | Suggested using the hardware's Event Task Matrix (ETM) via Remote Control Transceiver (RMT) or timers for sub-micros pulse delivery.<br><br>**Extra:**<br>Emphasized open-drain emulation and hardware clamping for I2C to let pins safely pull signals low without causing short circuits and protecting pins from dangerous voltage spikes respectively. | **<u>Weak Solution 2</u>**<br>**Solution:**<br>Suggested driving a push-pull high signal into a wired ESP32 victim UART transmitter (TX) line.<br><br>**Problem:**<br>Creates a dead short circuit. A dead short is where electrical current flows with zero resistance. Risks potential damage to electrical systems like silicon degradation. Why even??? | **For I2C ACK suppression:**<br>Configure the ESP pin connected to the victim SDA or SCL pin strictly as open-drain mode. Prevents dangerous voltage clashing. It is only allowed to pull the voltage down to low or let it float high naturally.<br><br>**Extra:**<br>Ensure fuzzer task Chaos_Fuzzer_Task acquired the hardware resource MTX_PROBE_ACCESS before arming. Prevents other areas of programming from interfering during the implementation. |
| Zayne Lee | Model: Sonnet 5.5<br><br>Prompt: "On an ESP32-C6 with uT-Kernel 3.0, we need to capture 802.11ax Wi-Fi management frames and IEEE 802.15.4 Zigbee frames simultaneously while hosting a local Web UI for telemetry. What are the RF architecture risks and how should we configure the promiscuous callback?" | Confirmed that ESP32-C6 shares a single 2.4 GHz RF front-end between Wi-Fi and 802.15.4, meaning true simultaneous physical capture is impossible. Recommended a deterministic time-multiplexed capture window.<br>* Warned against executing blocking calls inside the promiscuous callback. | Claimed ESP32-C6 can do simultaneous full-duplex dual-protocol capture using internal virtual MACs.<br>Suggested using standard FreeRTOS queues directly inside the ESP-IDF promiscuous callback to pass packets to uT-Kernel, which causes kernel context corruption because ESP-IDF Wi-Fi callbacks run in private ISR context.<br>* Suggested capturing raw BLE link-layer packets without hardware baseband support. | Formalize RF time-multiplexing: run SoftAP Web UI on Channel 6, and time-slice sniffing windows onto target channels (Wi-Fi Ch 1-13 or 802.15.4 Ch 11-26). Restrict promiscuous callback to non-blocking memory pool acquisition (`tk_tget_mpf` with `TMO_POL`) from `MPF_PACKETS`; if pool is full, increment drop counter and drop packet immediately rather than stalling radio ISR. |
| Ryan Ke | Model: DeepSeek V4.1<br><br>Prompt: We are running a wireless packet sniffer and we will be using ESP32-C6s as well as the Maker Pi Pico, with the ESP32 as the packet sniffer and also | - Suggested GPIO18/19/20/EXIO7 for pins as CLK/CMD/DAT0/DAT3<br><br>- Suggested using the file format of libpcap.<br><br>- Suggested write path through Radio ISR → | - Use the four probe lines GPIO4-7 as the SD SPI bus. Bit-bang SPI on any free GPIO and drive SD_CS directly from a GPIO. Use the UART debug pins GPIO16/17 (U0TXD/U0RXD) for SCK/MOSI. | - We can continue to use GPIO18/19/20/EXIO7 since those are specifically the pins we should be using for the SD card interface as stated in the datasheet.<br><br>- Reject large JSON outputs. Ideally we |

| | | | | |
|---|---|---|---|---|
| | the sender to the Maker Pi Pico base, which has the SD card base for writing.<br><br>What are the SPI pins I should be using to connect the SD card interface and how do I write the data format to disk? Additionally, what is this data format I should be using and the data write rate, as well as how I would sniff and send the data packets from the wireless line to the SD card interface? | MPF_PACKETS (8 KB fixed pool, no malloc) → MBF_STORAGE (message buffer) → Storage_Task → serialize → SPI DMA → SD, all at 512-byte sector-aligned chunks to amortize FAT/metadata overhead.<br><br>- Suggested the single-radio capture→SD path | - Write raw text JSON strings to the SD card, one object per captured packet. Log human-readable hex/CSV lines instead of a binary container.<br><br>- malloc a buffer per packet inside the Radio ISR. Write each packet straight to SD from the ISR (no queue). Write byte-by-byte / line-by-line, no sector alignment.<br><br>- Capture Wi-Fi 6 and 802.15.4 simultaneously on the one 2.4 GHz front-end. Time-multiplex the radio per packet between sniffing and telemetry. Stream every sniffed frame live to the dashboard over the same radio. | should use a lighter data format like binary libpcap, aligning with FR8 and use less resources.<br><br>- Reject dynamic malloc as it violates NFR5. We need 0-copy fixed pools Also reject ISR-direct writes as it can cause unexpected behavior, and cause unbuffered writes due to dropped frames.<br><br>- No, ESP32 WiFi module is half-duplex. Only transmit/receive happening serially. So we receive, capture goes to SD card via wired SPI with a custom 1 second buffer to avoid the 40ms x frames timeframe. |
| Joseph Poon | Model: GLM 5.3<br><br>Prompt: "How should the System Supervisor task on ESP32-C6 coordinate E-Stop, a local WebSocket dashboard, and peripheral status indicators under uT-Kernel 3.0, ensuring E-Stop response <= 5 ms and zero dynamic memory allocation?" | Recommended handling E-Stop as an edge-triggered hardware GPIO interrupt directly manipulating GPIO output-enable registers (`GPIO.enable_w1tc`) to float probe pins in < 200ns, completely bypassing RTOS scheduling latency. Recommended static buffer allocation for WebSocket frame encoder. | Suggested including an onboard 0.96 OLED display alongside the WebSocket dashboard. An OLED consumes 1 KB RAM, bus bandwidth, and introduces display-refresh blocking delays (> 20ms) that interfere with supervisor responsiveness. Suggested routing E-Stop through a normal RTOS message queue to be processed by a worker task, violating the <= 5ms safety preemption requirement. | Drop the OLED display; rely entirely on the WebSocket dashboard over Wi-Fi and onboard NeoPixel/buzzer. Implement E-Stop as a 2-stage mechanism: Stage 1 (Hardware ISR) directly clears `GPIO.enable_w1tc` in < 200ns to float all 4 probe channels to High-Z, then sets event flag `FLG_SAFETY_ABORT`; Stage 2 (`Safety_Supervisor_Task`, Priority 1) wakes, commands WS2812 status LED to RED, activates buzzer (`EXIO3`), and coordinates systematic task disarm within 2ms (<= 5ms bound). Ensure GPIO9 boot strapping |

| | | | | |
|---|---|---|---|---|
| | | | | pin is unpressed during power-up. |

## Week 6 Architecture Defence Snapshot Before Zoom

Select only **5 to 7** important Week 6 architecture claims that affect integration readiness. Do not repeat the entire Week 6 document. Each row should defend one claim using evidence, remaining weakness, and the buddy responsible. One smart-toaster example is shown; replace it with your own project claim.

| Week 6 artefact | What the team claimed in Week 6 | Evidence available now | Still weak or untested | Buddy defending it |
|---|---|---|---|---|
| Hardware Integration: Direct 3.3V Bus Coupling (NFR-4) | We eliminated TXS0108E bidirectional level shifter; coupled ESP32-C6 probes directly to RP2040 3.3V rails (3.3V +- 5%, V_abs <= 3.63V) with 220 ohm series resistor on Victim A TX line. | Voltage compliance verified against RP2040 datasheet (V_IO = 3.3V) and ESP32-C6 TRM; removing shifter eliminates 10ns propagation delay and open-drain rise-time distortion. 220 ohm resistor bounds push-pull short circuit current to <= 15mA. | Transient capacitive ringing during high-speed switching (> 1 MHz) across breadboard jumper wires not yet verified on high-bandwidth oscilloscope. | Annie Goh |
| Memory Management : Deterministic Fixed Pool (NFR-2 / NFR-3) | 0 dynamic heap allocations (`malloc`/`free`) during continuous capture and fault injection; all telemetry and packet serialization uses pre-allocated 32 KB Fixed Memory Pool `MPF_PACKETS` (16 blocks of 2048 B). | Week 6 memory map allocates `MPF_PACKETS` in BSS segment; diagnostic benchmark (`test1_chip_diagnostics.ino`) confirms 512 KB SRAM headroom with > 250 KB free heap remaining. | Sustained 10-min continuous stress test at 500 kbps to verify 0 pool exhaustion when MicroSD sector write latency spikes to 40ms. | Ryan Ke |
| Fault Injection Precision: Sub-Microsecond Glitch Timing (NFR-5) | Fault injection pulses synchronize to target bus trigger edges with timing jitter < 500ns, completely bypassing RTOS task scheduling jitter. | Hardware Event Task Matrix (ETM) connects GPIO comparator directly to RMT TX channel; hardware trigger path operates within 3 to 5 clock cycles (< 50ns at 160 MHz). | Jitter under heavy radio DMA traffic (concurrent Wi-Fi promiscuous RX) not yet empirically measured on physical digital storage oscilloscope. | Annie Goh |
| Safety Architecture: E-Stop Hardware Preemption <= 5ms (NFR-1) | Emergency stop on GP12 (`GPIO9` BOOT switch) preempts all background processing, floating all 4 probe channels to High-Z within < 200ns (ISR) and completing system-wide alarm and task disarm within <= 5ms. | Stage 1 ISR directly writes to 32-bit register `GPIO.enable_w1tc`, taking < 50ns; `Safety_Supervisor_Task` runs at highest system priority (Priority 1). | Verifying that pressing E-Stop during power-on reset does not hold GPIO9 low and accidentally force ESP32-C6 into ROM UART bootloader mode. | Joseph Poon |

| | | | | |
|---|---|---|---|---|
| Bus Auto-Discovery: Hardware RMT RX Offload (FR-1 / FR-2) | Passive auto-discovery accurately identifies UART (9600-115200 baud, +- 2%) and I2C (100/400 kHz, +- 2%) within <= 500ms across 4 pins without CPU interrupt thrashing. | ESP32-C6 RMT peripheral has 4 RX channels capable of streaming pulse duration items directly to RAM via DMA without per-edge CPU interrupts. | Accuracy of statistical baud clustering in the presence of bus noise, reflections, or flat-line idle states during bus reset. | Tan Yan Qi |
| Wireless Architecture: Time-Multiplexed 2.4 GHz RF Front-End (FR-4 / Assumption A4) | The single 2.4 GHz RF transceiver operates in distinct, time-multiplexed capture modes (Wi-Fi 6 promiscuous Ch 1-13 vs IEEE 802.15.4 Zigbee Ch 11-26), while the Web UI dashboard runs on Home Channel 6. | ESP32-C6 RF architecture documentation confirms shared baseband; firmware state machine implements dedicated `WIFI_SNIFF` and `ZIGBEE_SNIFF` states. | Packet loss rate of WebSocket telemetry updates during channel-hopping sweeps across distant Wi-Fi channels (eg Ch 1 or Ch 13). | Zayne Lee |

## Shared Resource Allocation Matrix

Enter at least **3 shared resources**, or more if the project has additional shared buses, timers, interrupts, actuators, sensors, queues, event groups, or shared data. A resource is “shared” if more than one task, module, interrupt, or buddy can affect it.

| Shared resource | Used by task(s) | Owner task / module | Access type | Contention risk | Mitigation | Evidence needed |
|---|---|---|---|---|---|---|
| Probe GPIO Bus (GP4-GP7 / GPIO6, 7, 14, 15) | `AutoDiscovery_Task`, `Chaos_Fuzzer_Task`, `Safety_Supervisor_Task` | `AutoDiscovery_Task` (Default), `Chaos_Fuzzer_Task` (Armed) | Time-exclusive; High-Z read default; Transient low drive during glitch | Fuzzer drives pin while Auto-Discovery is measuring edge timing, corrupting baud estimation or creating bus contention. | Protected by uT-Kernel mutex `MTX_PROBE_ACCESS`. Fuzzer must acquire mutex before arming. E-Stop ISR overrides via direct register write. | DSO capture showing probe pins cleanly release to High-Z before fuzzer arming; 0 mutex deadlocks over 100 contention cycles. |
| Onboard MicroSD Card SPI Bus (GPIO10, 11, 12, 15 / Pins 14-16, 20) | `Storage_Task` exclusively; inputs fed from `Wireless_Engine` & `AutoDiscovery` | `Storage_Task` (`TSK_STORAGE`) | Dedicated master SPI bus; block sector writes (512 B) | Slow SD card flash write latency spikes (<= 40ms) block high-speed packet ingestion tasks. | Decoupled by message buffer `MBF_STORAGE` (depth 16) and zero-copy fixed memory pool `MPF_PACKETS`. Producer tasks enqueue pointers; `Storage_Task` drains at Priority 5. | Benchmark showing 0 dropped packets during simulated 40ms SD card flash block-erase stall. |

| | | | | | | |
|---|---|---|---|---|---|---|
| 2.4 GHz RF Transceiver Baseband | `Wireless_Engine_Task`, `Web_Dashboard_Task` | `Wireless_Engine_Task (TSK_WIRELESS)` | Time-multiplexed RF front-end (RX promiscuous vs TX SoftAP beacon/data) | SoftAP telemetry broadcasts on Channel 6 interrupted or dropped when radio hops to Channel 1 or Zigbee Channel 15 to capture packets. | Time-sliced buffering: sniffer captures on target channel for 1 to 2s into memory pool, then returns to Channel 6 to flush telemetry over WebSocket. | RF channel dwell timer logs showing clean return to Channel 6 without WebSocket client disconnection. |
| Fixed Packet Memory Pool (`MPF_PACKETS`, 32 KB / 16 Blocks of 2048 B) | `Wireless_Engine_Task`, `AutoDiscovery_Task`, `Storage_Task`, `Web_Dashboard` | `Kernel Object (MPF_PACKETS), allocated at system boot` | Zero-copy pointer exchange across task message queues | Memory pool exhaustion during burst traffic leads to task deadlock or packet dropping. | Non-blocking acquisition `tk_tget_mpf(..., TMO_POL)`. If pool exhausted, drop packet and increment drop counter; never block radio ISR. Consumers release via `tk_rel_mpf`. | Stress test verifying pool block count never leaks over 10,000 allocate/release cycles. |
| Onboard I2C Expansion Bus (TCA9554 PWR on GPIO22/GPIO23) | `Safety_Supervisor_Task` exclusively | `Safety_Supervisor_Task (TSK_SAFETY)` | Master I2C write to control Piezo Buzzer (`EXIO3`) and status LEDs | I2C bus hang or arbitration loss if another task attempts to probe expander pins while safety buzzer is being commanded. | Strictly isolate TCA9554PWR I2C bus to `Safety_Supervisor_Task`. Baseboard pushbuttons 1 & 2 (GP20/GP21) are strictly banned from software use to prevent shorting expander I2C lines to ground. | I2C bus analyzer log confirming 0 unauthorized transactions on address `0x20` (TCA9554). |

## Subsystem Interface and Code Structure Map

Enter at least **one interface or module per buddy**. Each row should identify who provides the module, who consumes it, what inputs and outputs it exposes, and how it will be validated.

| Module / file / interface | Provider | Consumer(s) | Purpose | Inputs | Outputs | Interface rule | Validation rule |
|---|---|---|---|---|---|---|---|
| `auto_discovery.c probe_sample_edges_rmt()` | Tan Yan Qi | `AutoDiscovery_Task`, `Chaos_Fuzzer_Task`, `Web_Dashboard_Task` | Samples edge transition timestamps from 4 probe channels via RMT | RMT RX DMA raw buffer (pulse item stream, microsecond | `proto_desc_t` (Protocol: UART/I2C /IDLE, Baud rate, Pin roles: | Only `AutoDiscovery_Task` executes baud classification; emits event flag | Pass if UART (9600-115200 baud, +- 2%) and I2C (100/400 kHz, +- 2%) |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | RX, computes pulse statistics, and resolves active bus pinout. | timestamps) | TX, RX, SDA, SCL) | `FLG_PROTO_LOCKED` upon stable identification (>= 95% confidence over 500ms). | correctly identified within <= 500ms on test bench. |
| `chaos_fuzzer.c` `fuzzer_invert_uart_bit()` `fuzzer_ack_suppress_i2c()` | Annie | `Chaos_Fuzzer_Task`, Target Victims (Pico A & Pico B) | Injects deterministic, single-bit pulse glitches (< 500ns jitter) or pulls I2C SDA low during 9th clock cycle to force NACK. | Fuzz target trigger pattern, delay offset, pulse duration (<= 10us, duty cycle <= 5%) | Timed GPIO pulse on probe line via hardware ETM / RMT TX channel | Must acquire `MTX_PROBE_ACCESS` before configuring ETM trigger; immediately float pin to High-Z upon completion or E-Stop event flag. | Oscilloscope capture showing edge-to-pulse jitter < 500ns, current draw <= 15mA through 220 ohm resistor, and victim reporting framing error/NACK. |
| `wireless_sniffer.c` `wifi_promiscuous_rx_cb()` `zigbee_rx_cb()` | Zayne | `Wireless_Engine_Task`, `Storage_Task`, `Web_Dashboard_Task` | Captures raw over-the-air 802.11ax and 802.15.4 packets, filters headers, extracts RSSI, and passes packets to storage buffer. | Raw radio baseband FIFO interrupts and frame buffers | Serialized packet buffer pushed to `MBF_STORAGE` and `MBF_TELEMETRY` | Promiscuous callback must execute in < 5us; acquires buffer via `tk_tget_mpf(..., TMO_POL)`; drops frame if pool is full; never blocks. | Serial log confirming <= 0.1% frame drop rate during 100 pkt/s beacon burst; Wireshark validates captured PCAP checksums. |
| `pcap_storage.c` `pcap_write_frame()` `sd_flush_sector()` | Ryan | `Storage_Task`, MicroSD SPI Driver | Formats captured packets with 16-byte `libpcap` header, stages data into 512-byte sector cache, and flushes to | Packet descriptor from `MBF_STORAGE`, microsecond timestamp, length, raw bytes | 512-byte block SPI commands to MicroSD card (`/sdcard/capture.pcap`) | All writes are sector-aligned; 0 dynamic allocations; releases packet buffer back to `MPF_PACKETS` via `tk_rel_mpf` immediately after sector staging. | MicroSD card mounted on PC opens in Wireshark with 0 corrupted headers or malformed length fields. |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | FAT32 MicroSD card. | | | | |
| `supervisor_safety.c` `safety_estop_isr()` `ws_broadcast_telemetry()` | Joseph | `Safety_Supervisor_Task`, All system tasks, Web UI clients | Coordinates system health monitoring, executes hardware E-Stop probe disarm, and streams live 1 to 2 Hz JSON telemetry over WebSocket. | GPIO9 switch interrupt, system task health flags, telemetry queue `MBF_TELEMETRY` | `GPIO.enable_w1tc` 32-bit register clear, WS2812 LED color, buzzer output, WebSocket JSON frames over Wi-Fi | Stage 1 E-Stop executes directly in ISR without RTOS scheduling; WebSocket transmission drops telemetry frames if client socket buffer is congested. | Oscilloscope confirms probe pin floats to High-Z within < 200ns of E-Stop press; dashboard displays live packet rate at 1 to 2 Hz +- 10%. |

## Week 6 Assumption Closure

Enter at least **one assumption per buddy**, preferably copied from Week 6. Each row should show whether the assumption is still open, validated, deferred, or converted into a risk.

| Week 6 assumption ID | Original assumption | Current status | Evidence collected | Remaining risk | Next action |
|---|---|---|---|---|---|
| A0 (Vendor Coexistence) | uT-Kernel 3.0 and ESP-IDF closed-source binary drivers (Wi-Fi/802.15.4 blobs) can coexist via the FreeRTOS translation shim without memory corruption. | Needs Validation | FreeRTOS translation shim in `esp32c6-mtk3` compiles; standalone benchmark test (`test1_chip_diagnostics.ino`) validates basic RTOS boot. | Vendor Wi-Fi blob may invoke un-shimmed FreeRTOS primitives at runtime, causing silent memory corruption or kernel panic. | Flash test firmware with promiscuous RX enabled; verify 30-min run without panic before Week 8 demo. Owner: Zayne |
| A1 (Direct 3.3V Driving) | ESP32-C6 GPIO pins can switch from High-Z Input to Driven Output in < 1us for targeted bit glitching without damaging target microcontrollers. | Validated / Closed | Integrated 220 ohm series resistor limits short-circuit current to <= 15mA; pulse duration clamped to <= 10us (<= 5% duty cycle); open-drain mode used on I2C. | Minor capacitive rise time degradation on breadboard wiring (~15ns). | Probe edge slew rate on physical DSO during Week 8 lab session. Owner: Annie |

| | | | | | |
|---|---|---|---|---|---|
| A2 (Buffer Overrun Risk) | ESP32-C6 promiscuous packet filter captures raw 802.11 management frames without dropping frames at burst rates up to 50 to 100 pkts/s. | Validated / Closed | Fixed memory pool `MPF_PACKETS` upsized from 8 KB to 32 KB (16 blocks of 2048 B), absorbing burst traffic even when MicroSD write stalls for 40ms. | Flash write latency exceeding 100ms on degraded/unbranded SD cards. | Mandate SanDisk Extreme Class 10 U3 MicroSD card; measure sector write latency distribution. Owner: Ryan |
| A3 (Deauth Containment) | Targeted deauth injection will be contained strictly to isolated testbed victim node without spilling into campus Wi-Fi. | Validated / Closed | Hardware BSSID/MAC allow-list (`02:00:00:00:00:03`) hardcoded in firmware; broadcast deauth (`FF:FF:FF:FF:FF:FF`) strictly prohibited in software state machine. | User accidentally modifying target MAC address in web dashboard. | Enforce firmware validation check that rejects any MAC outside the testbed namespace. Owner: Zayne |
| A4 (Single RF Multiplexing) | Single 2.4 GHz RF front-end can switch between Wi-Fi and Zigbee without crashing the SoftAP telemetry server. | Converted to Risk / Mitigated | Time-sliced capture architecture documented in Section 2; dashboard operates on Channel 6, sniffer sweeps remote channels and buffers data locally before returning to Channel 6. | WebSocket client may experience momentary 1-second packet delivery latency during channel hop. | Implement client-side WebSocket heartbeat reconnection handling in HTML5 dashboard. Owner: Joseph |
| A5 (Auto-Discovery Accuracy) | Passive 4-pin auto-discovery edge timing can classify UART and I2C within 500ms without false locks. | Open / In Progress | RMT RX DMA driver implemented; timing threshold tables defined (8.68us +- 2% for 115200 baud; 2.5us +- 2% for 400 kHz I2C). | High bus noise or floating pins could trigger spurious edge transitions before protocol lock. | Integrate 3-stage debounce filter and require at least 16 valid consecutive edge rhythms before emitting `FLG_PROTO_LOCKED`. Owner: Tan Yan Qi |
| A6 | Hardware E-Stop button on GPIO9 can preempt all | Validated / Closed | ISR writes to `GPIO.enable_w1tc` in < 200ns; | User accidentally holding E-Stop | Add power-on warning LED pattern in |

| | | | | | |
|---|---|---|---|---|---|
| (E-Stop & GPIO9 Invariant) | tasks within 5ms while maintaining boot stability. | | operational invariant established requiring GPIO9 to remain unpressed during the first 50ms power-on reset. | during board power-up, causing ESP32-C6 to enter ROM download mode. | bootloader and document requirement in physical testbed setup guide. Owner: Joseph |
| A7 (GPIO15 JTAG Strapping) | PROBE_3 on GPIO15 (Maker Pi GP7) can be used as a probe without interfering with JTAG debug or board reset. | Validated / Closed | GPIO15 is kept in High-Z floating mode during boot; verified ESP32-C6 boot log via UART0 console shows normal flash boot into application. | Victim pulling line LOW during reset could divert JTAG lines. | Connect I2C SCL to GP7 only after boot confirmation or use internal pull-up. Owner: Tan Yan Qi |

## Version B: Live Zoom-Updated Version

### Instructions

- During the recorded Zoom session, update this section live or immediately from discussion notes.
- Record what changed from Version A and why. A change without justification is weak evidence.
- Every buddy must contribute at least one challenge, defence, trade-off, or correction.
- Any use of AI during the Zoom session must be screen-shared and visible to all participants at the point it is being discussed.
- If AI-generated content is introduced during the live discussion, the team must explain the prompt used, the output received, and why the output is being considered.
- AI suggestions must be challenged, validated, refined, or rejected through human engineering discussion; “AI said so” is not sufficient justification.

**Acceptable evidence examples:** GitHub commit link, short test log, video timestamp, oscilloscope or serial monitor screenshot, pin test result, task timing measurement, AI prompt and critique, meeting decision note, or photo of hardware wiring.

**AI-use declaration:** Do not ask AI to complete this template on behalf of the team. Use AI as a reviewer, challenger, or brainstorming partner. The team confirms that all final entries were reviewed, corrected, and accepted by the team, and that every member can explain the decisions related to their subsystem.

## Buddy Contributions During Zoom

Each buddy must complete **one row**. The contribution must be a real technical challenge, defence, trade-off, or correction raised during Zoom, not only an attendance statement.

| Buddy | Key challenge, concern, or argument raised | Challenge received | Decision or compromise reached |
|---|---|---|---|
| Buddy 1 | | | |
| Buddy 2 | | | |
| Buddy 3 | | | |

| | | | |
|---|---|---|---|
| Buddy 4 | | | |
| Buddy 5 | | | |

## Live Decision Change Log

Enter at least **3 changes AND 3 confirmed no-change decisions** from the Zoom discussion. Include the change type and evidence link or timestamp whenever possible. One row may state “No change” if the team defended the Week 6 design successfully.

| Original idea / Week 6 design | Updated decision | Change type | Reason for change | Evidence link / timestamp | Owner / follow-up |
|---|---|---|---|---|---|
| | | No change / Clarified / Simplified / Reassigned / Deferred / Rejected / Major redesign | | | |
| | | No change / Clarified / Simplified / Reassigned / Deferred / Rejected / Major redesign | | | |
| | | No change / Clarified / Simplified / Reassigned / Deferred / Rejected / Major redesign | | | |

## Resource and Assumption Review

| Item challenged | Evidence presented | Decision made | Impact on other buddies |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

## Risk Assessment and Subsystem Test Plan

### Integration Risk Register

Enter at least **3 integration risks**. Include at least one timing or scheduling risk, one hardware or wiring risk, and one interface or team-integration risk.

| Risk ID | Risk | Cause | Impact | Likelihood | Severity | Mitigation | Owner | Status |
|---|---|---|---|---|---|---|---|---|
| R1 | Example: Safety Monitor does not disable heater quickly enough. | Safety event is queued behind display or cooking updates. | Heater may remain on briefly during door-open or over-temperature condition. | Medium | High | Give Safety Monitor highest priority and test forced shutdown while other tasks are running. | | Open |
| R2 | | | | | | | | |

### Subsystem Test Plan

Enter **at least 4 cross-subsystem tests**. Each test must involve the work of **at least two buddies** and must include a success criterion and evidence type, such as a video, serial log, screenshot, timing result, or commit link. At least **TWO** test should involve timing, safety, or shared-resource concern.

| Subsystem | Test case | Success criteria | Evidence required | Owner | Status |
|---|---|---|---|---|---|
| Example: Safety Monitor | Open door switch while cooking is active. | Heater output turns OFF and system enters Fault within the stated response time. | Video plus serial log showing door-open event, heater OFF command, and Fault state. | Buddy 4 | Open |
| Example: sensor input | Detect line, barcode, or obstacle. | Correct reading appears in serial log. | Screenshot or log. | Buddy 3 | Open |
| | | | | | |

## Constraint Injection / Curveball Response

**Instructions**

- Record the injected constraint exactly as given.
- Explain how it affects all five buddies, even if the impact differs by subsystem.
- A good response shows prioritisation, trade-off reasoning, fallback behaviour, and evidence-aware decision making.

| Injected constraint | Buddy affected | Immediate impact | Adaptation / compromise |
|---|---|---|---|
| | Buddy 1 | | |
| | Buddy 2 | | |
| | Buddy 3 | | |
| | Buddy 4 | | |
| | Buddy 5 | | |

## Integration Readiness Self-Assessment

Complete all rows. Use **Green** only if there is evidence, **Yellow** if partly ready or untested, and **Red** if the area is unclear or blocking Week 10 integration.

| Readiness area | Status | Evidence | Concern |
|---|---|---|---|
| Hardware wiring known | Green / Amber / Red | | |
| RTOS task ownership clear | Green / Amber / Red | | |
| Shared resources identified | Green / Amber / Red | | |
| Critical statecharts implementable | Green / Amber / Red | | |
| Safety or fault handling defined | Green / Amber / Red | | |
| Week 10 partial integration feasible | Green / Amber / Red | | |

**Updated readiness position before the Week 10 partial integration demo:**

_____

_____

_____

## Version C: Final AI-Vetted Gap Check

### Instructions

- After the live discussion, give the updated Version B content to AI and ask AI to act as a strict embedded systems reviewer.
- Ask AI to identify missing assumptions, weak evidence, unclear interfaces, untested failure modes, and resource conflicts.
- For every AI-identified gap, decide whether to fix, reject, or defer it, and explain why.

| AI reviewer prompt used | |
|---|---|
| | |

| AI-identified gap | Team response | Action taken | Evidence / reason |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

## 5. Final Tribunal Summary

| Final question | Team answer |
|---|---|
| What is the strongest part of the current design? | |
| What is most likely to fail? | |
| What evidence supports readiness? | |
| What did AI miss that human discussion revealed? | |
| What did AI correctly identify after discussion? | |